

TUGAS KECIL 3
IF2211 STRATEGI ALGORITMA
Ice Sliding Puzzle Solver



Anggota:

Fauzan Mohamad Abdul Ghani	13524113
Muh. Hartawan Haidir	13524147

TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2025/2026

BAB I PENGGUNAAN ALGORITMA PATHFINDING

- **Algoritma Uniform-Cost Search (UCS)**

Algoritma Uniform-Cost Search (UCS) adalah algoritma pencarian graf (uninformed search) yang mencari rute dengan total biaya (bobot) terendah dari simpul awal ke tujuan. Algoritma UCS menggunakan priority queue untuk mengekskansi simpul berdasarkan akumulasi biaya terkecil, menjamin solusi optimal selama biaya langkah tidak negatif. Berikut adalah karakteristik yang lebih menggambarkan tentang algoritma UCS :

- **Fokus kepada biaya** : berbeda dengan algoritma BFS yang mencari jarak terpendek berdasarkan jumlah langkah, UCS memperhitungkan jarak berdasarkan total bobot/biaya yang didapat selama melangkah (contoh: jarak, waktu, biaya tol)
- **Optimalitas** : Algoritma UCS menjamin hasil optimal (jalur dengan biaya terendah) asalkan biaya yang diambil di setiap langkah tidak negatif.
- **Priority Queue** : Node yang akan dieksplorasi disimpan dalam priority queue dan selalu mengambil node dengan biaya kumulatif terendah.

Untuk lebih jelasnya, berikut adalah prosedur atau langkah dalam menjalankan algoritma Uniform-Cost Search (UCS) :

1. Hal yang paling pertama adalah Memasukkan simpul pertama atau simpul awal ke dalam priority queue
2. Kemudian, akan dilakukan pengecekan terhadap jalan yang bisa dilalui beserta biaya untuk perjalanannya. Algoritma akan mengambil simpul dengan biaya terendah untuk diekskansi
3. Jika node tujuan telah ditemukan, algoritma akan berhenti untuk mencari jalur lain, kemudian akan dihasilkan jalur yang telah dilalui sampai ke tujuan
4. Jika belum ditemukan, tetangga dari simpul tersebut akan ditambahkan ke priority queue dengan akumulasi biaya.

Contoh penerapan yang sering digunakan dalam algoritma UCS adalah aplikasi pemetaan, rute terpendek, atau sistem rekomendasi yang melibatkan optimasi biaya.

- **Algoritma Greedy Best-First Search (GBFS)**

Algoritma Greedy Best-First Search (GBFS) adalah algoritma pencarian terinformasi (informed search) yang mencari rute tercepat menuju tujuan dengan selalu memilih node yang terlihat paling dekat dengan tujuan berdasarkan fungsi heuristik, tanpa mempertimbangkan biaya yang sudah dikeluarkan sebelumnya. Berdasarkan fungsi heuristik $f(n) = h(n)$, algoritma akan memperkirakan

biaya dari node saat ini (n) ke node tujuan. Berikut adalah karakteristik yang lebih menggambarkan tentang algoritma GBFS :

- Algoritma GBFS mengambil keputusan terbaik secara lokal pada setiap langkah (rakus), dengan harapan hawa keputusan mengarah kepada solusi global yang paling optimal, namun tidak menjamin hasil optimal
- Algoritma GBFS memiliki keunggulan yang tidak dimiliki oleh algoritma lain, yaitu lebih cepat mencapai tujuan pada banyak kasus karena hanya berfokus pada arah tujuan
- Akan tetapi, algoritma GBFS bisa terjebak ke dalam siklus nya sendiri dan memungkinkan untuk menjadi tidak optimal (bisa menemukan jalur yang lebih panjang daripada yang terpendek)

Algoritma GBFS bekerja dengan cara melakukan evaluasi terhadap semua node yang berhubungan atau bertetangga dengannya, kemudian memiliki salah satu yang mempunyai nilai $h(n)$ terkecil (jarak perkiraan terdekat) untuk dieksplorasi lebih lanjut

Algoritma ini hampir mirip dengan Breadth-First Search (BFS) dan Depth-First Search (DFS), namun lebih ke arah menggunakan antrian berprioritas (priority queue) berdasarkan nilai heuristik

● **Algoritma A***

Algoritma A* adalah salah satu algoritma pencarian jalur (pathfinding) yang efisien dan bisa dikatakan sebagai algoritma paling populer dalam kecerdasan buatan (AI) dan Ilmu Komputer. A* dapat menemukan jalur terpendek seperti algoritma yang lain, tetapi lebih cepat dan optimal. Algoritma A* juga merupakan peningkatan dari algoritma Greedy Best-First Search dan Dijkstra atau merupakan penggabungan dari keduanya. Untuk lebih jelasnya, berikut adalah keunggulan algoritma A* dibanding yang lain :

- Algoritma A* dapat menemukan jalur terpendek dengan waktu komputasi yang lebih cepat daripada algoritma pencarian buta karena menggunakan heuristik untuk memandu pencarian
- Selain itu, algoritma ini dapat menjamin hasil yang optimal

A* akan menggunakan daftar Open List dan Closed List untuk menyimpan node yang akan diperiksa dan node yang sudah diperiksa. Untuk lebih jelasnya, berikut adalah langkah-langkah dalam algoritma A* :

1. Tambahkan titik awal ke Open List.
2. Ulangi langkah berikut hingga tujuan ditemukan atau Open List kosong:
 - a. Cari node di Open List dengan nilai $f(n)$ terkecil.
 - b. Pindahkan node tersebut dari Open List ke Closed List.

- c. Periksa tetangga dari node tersebut. Jika tetangga adalah tujuan, jalur ditemukan.
- d. Jika tetangga belum ada di Open List/Closed List, hitung $f(n)$ nya dan tambahkan ke Open List.
- e. Jika tetangga sudah di Open List tetapi jalur baru lebih murah, perbarui nilai ($f(n)$) dan parent node nya.

Algoritma A* dalam digunakan di berbagai pekerjaan, seperti Game Development (menemukan jalur karakter (NPC) dan menghindari rintangan), Peta Digital (mencari rute terpendek antar kota atau lokasi), atau robotika (perencanaan jalur pergerakan robot).

BAB II ANALISIS ALGORITMA

2.1 Definisi $f(n)$ dan $g(n)$ pada setiap Algoritma

Dalam game Ice Sliding Puzzle, setiap gerakan pin menghasilkan cost sebesar jumlah cost tile yang dilewati selama satu gerakan sliding. Berikut definisi $g(n)$, $h(n)$, dan $f(n)$ pada setiap algoritma:

Algoritma	$g(n)$	$h(n)$	$f(n)$
UCS	Total cost dari state awal ke state n	Tidak digunakan	$f(n) = g(n)$
GBFS	Tidak digunakan	Estimasi jarak dari n ke goal	$f(n) = h(n)$
A*	Total cost dari state awal ke state n	Estimasi jarak dari n ke goal	$f(n) = g(n) + h(n)$

Keterangan:

- $g(n)$: cost aktual yang telah dikeluarkan dari state awal hingga state n . Dihitung sebagai penjumlahan cost seluruh tile yang dilalui di setiap gerakan sliding.
- $h(n)$: estimasi biaya dari state n menuju goal. Bersifat admissible jika tidak pernah melebihi-lebihkan biaya sebenarnya.
- $f(n)$: nilai prioritas yang digunakan priority queue untuk menentukan node mana yang diekspansi berikutnya.

2.2 Admissibility Heuristik pada A*

Sebuah heuristik dikatakan admissible jika selalu memenuhi kondisi $h(n) \leq h^*(n)$ untuk semua state n , di mana $h^*(n)$ adalah cost sesungguhnya dari n ke goal. Berikut analisis ketiga heuristik yang diimplementasikan:

1. H1, Manhattan Distance

Rumus: $\text{abs}(\text{state.row} - \text{goal_row}) + \text{abs}(\text{state.col} - \text{col_goal})$

Pada Ice Sliding Puzzle, satu gerakan sliding dapat melewati beberapa tile sekaligus. Nilai Manhattan distance mengukur jumlah minimum perpindahan baris dan kolom secara terpisah. Karena pin harus melakukan setidaknya satu gerakan untuk setiap dimensi yang perlu dijangkau, Manhattan, Manhattan distance tidak pernah melebihi-lebihkan jumlah gerakan yang dibutuhkan, berarti heuristik ini Admissible.

2. H2, Chebyshev Distance

Rumus: $\max(\text{abs}(\text{state.row} - \text{goal_row}), \text{abs}(\text{state.col} - \text{goal_col}))$

Chebyshev distance selalu $\leq$ Manhattan distance, sehingga jika Manhattan admissible maka Chebyshev juga admissible. Heuristik ini lebih optimis dan dapat membuat A* mengeksplorasi lebih sedikit node, berarti heuristik ini Admissible.

3. H3, Euclidean Distance

Rumus: $\sqrt{(state.row - goal_row)^2 + (state.col - goal_col)^2}$

Euclidean distance mengukur jarak garis lurus antara state dan goal. Nilainya selalu $\leq$ Manhattan distance, sehingga tidak pernah overestimate, berarti heuristik ini Admissible.

2.3 Perbandingan Teoritis Algoritma

Berikut perbandingan ketiga algoritma secara teoritis dalam konteks Ice Sliding Puzzle:

Kriteria	UCS	GBFS	A*
Optimalitas	Ya (selalu)	Tidak	Ya (jika h admissible)
Kelengkapan	Ya	Terbatas	Ya
Kompleksitas Waktu	$O(b^{(C^*/e)})$	$O(b^m)$	$O(b^d)$
Kompleksitas Ruang	$O(b^{(C^*/e)})$	$O(b^m)$	$O(b^d)$
Kecepatan	Lambat	Cepat	Seimbang
Pakai Heuristik	Tidak	Ya	Ya

Keterangan: b = branching factor, d = kedalaman solusi, m = kedalaman maksimum, C^* = biaya solusi optimal, e = biaya minimum per langkah.

Pada Ice Sliding Puzzle, branching factor maksimum adalah 4 (U/D/L/R). Namun karena banyak gerakan yang tidak valid (tabrak dinding, lava, keluar batas), branching factor efektif biasanya lebih kecil. A* dengan heuristik yang baik cenderung mengeksplorasi node yang sedikit dan menemukan solusi optimal paling cepat.

BAB III SOURCE PROGRAM

3.1 Struktur Modul

Program dibangun secara modular dengan pemisahan tanggung jawab yang jelas. Setiap file hanya mengandung satu tanggung jawab sehingga mudah dipahami, diuji, dan dimodifikasi secara independen.

File	Tanggung Jawab
models.py	Definisi struktur data: Board, State, SlideResult, SearchResult
parser.py	Membaca dan memvalidasi file input.txt
game.py	Logika mekanik sliding: fungsi slide() dan konstanta arah
heuristic.py	Fungsi heuristik H1(Manhattan), H2(Chebyshev), H3 (Euclidean)
solver.py	Algoritma pencarian: UCS, GBFS, A* dalam satu fungsi generik
visualizer.py	Tampilan papan di terminal
playback.py	Playback interaktif keyboard dan penyimpanan solusi ke .txt
main.py	Entry point: koordinator alur input/output pengguna
gui.py	Menampilkan gui program

3.2 Penjelasan Fungsi Utama

Fungsi slide() di game.py mensimulasikan satu gerakan sliding pin dari posisi (row, col) ke arah (dr, dc). Pin terus meluncur sampai menabrak dinding (X), mencapai tujuan (O), atau terjadi kondisi game over. Fungsi mengembalikan SlideResult dengan valid=False jika gerakan tidak sah.

Fungsi search() mengimplementasikan ketiga algoritma dalam satu fungsi generik. Perbedaannya hanya pada fungsi compute_f() yang menghitung prioritas node:

- UCS: $f(n) = g(n)$, hanya cost aktual
- GBFS: $f(n) = h(n)$, hanya heuristik
- A*: $f(n) = g(n) + h(n)$, gabungan keduanya

Fungsi menggunakan heapq (min-heap) sebagai priority queue dan dictionary best_cost untuk mencatat cost terbaik tiap state, mencegah eksplorasi berulang.

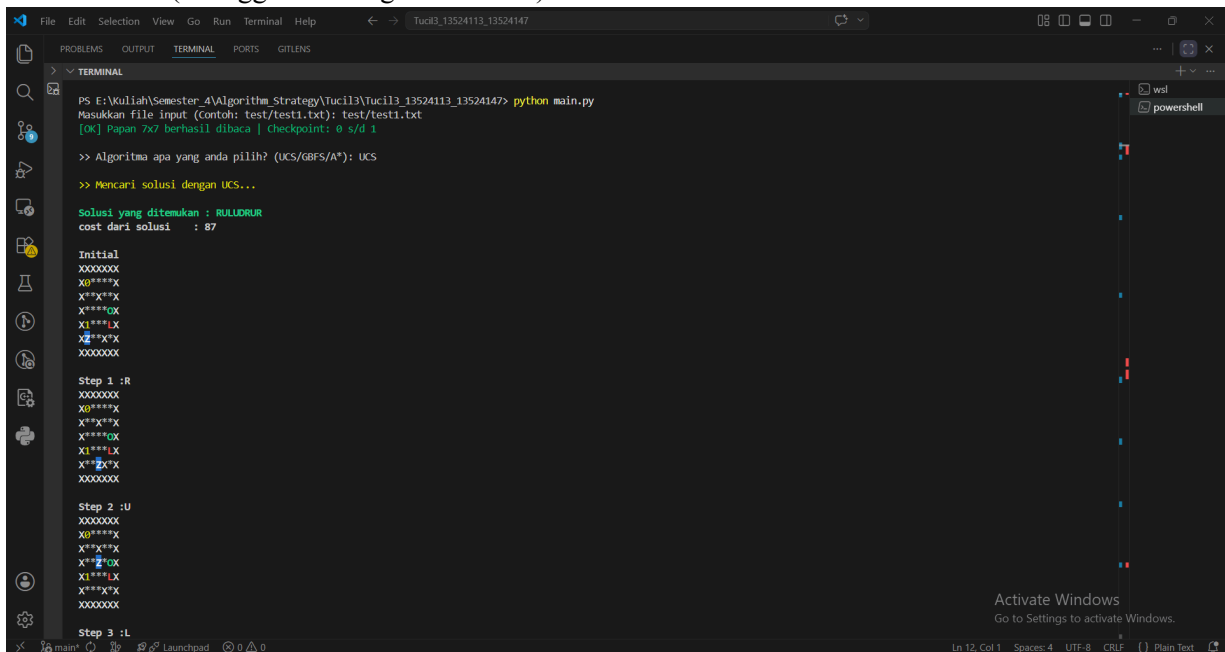
State direpresentasikan sebagai tuple (row, col, next_checkpoint) menggunakan Python dataclass dengan frozen=True. Atribut next_checkpoint diperlukan karena posisi pin saja tidak cukup untuk mendeskripsikan state secara unik, progress checkpoint menentukan gerakan apa yang valid ke depannya.

BAB IV INPUT DAN OUTPUT

1. Test Case 1

```
7 7
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999
```

TERMINAL (menggunakan algoritma UCS)



```
PS E:\Kuliah\Semester 4\Algorithm Strategy\Tucil3\Tucil3_13524113_13524147> python main.py
Masukkan file input (Contoh: test/test1.txt): test/test1.txt
[OK] Papan 7x7 berhasil dibaca | checkpoint: 0 s/d 1

>> Algoritma apa yang anda pilih? (UCS/GBFS/A*): UCS

>> Mencari solusi dengan UCS...

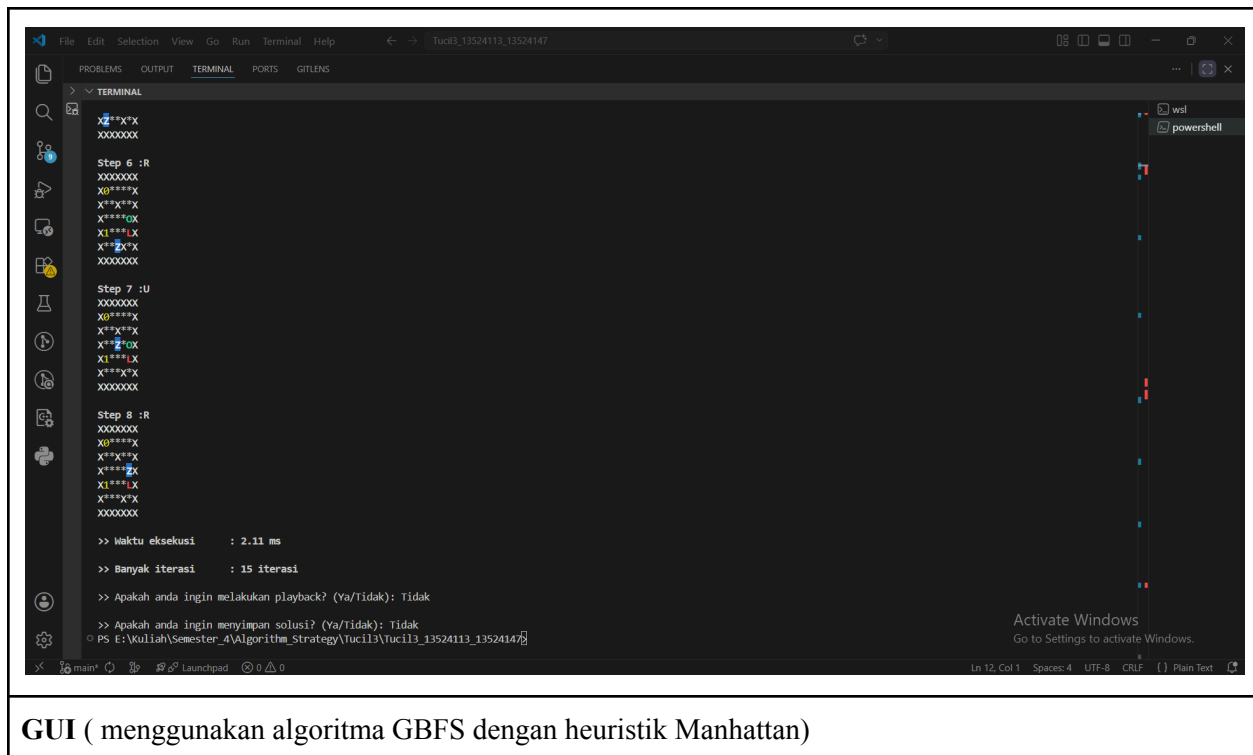
Solusi yang ditemukan : RULLDRUR
cost dari solusi      : 87

Initial
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Step 1 :R
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Step 2 :U
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX

Step 3 :L
```



GUI (menggunakan algoritma GBFS dengan heuristik Manhattan)

X	X	X	X	X	X	X
X	0	*	*	*	*	X
X	*	*	X	*	*	X
X	*	*	*	*	P	X
X	1	*	*	*	L	X
X	Z	*	*	X	*	X
X	X	X	X	X	X	X

Algoritma

- ☐ UCS
☒ GBFS
☐ A*

Heuristic

- ☒ H1 - Manhattan
☐ H2 - Chebyshev
☐ H3 - Euclidean

Load Map

Run

Informasi Solusi

Cost: 87
Waktu: 0.24 ms
Iterasi: 15
Solusi: R → U → L → U → D → R → U → R

Export Solution

Playback Solusi

Prev Next Jump
Play Pause Stop

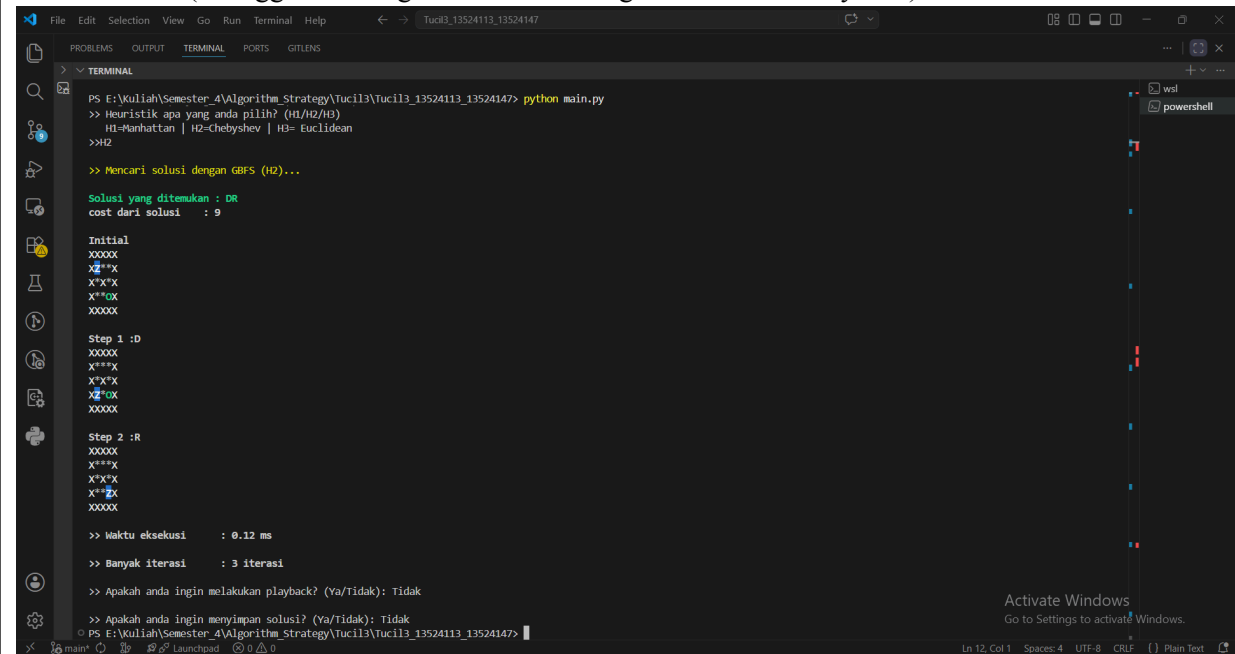
Speed (ms): 500

Step: 8/8

2. Test Case 2

```
5 5
XXXXX
XZ**X
X*X*X
X**OX
XXXXX
999 999 999 999 999
999 1 2 3 999
999 4 999 2 999
999 1 3 1 999
999 999 999 999 999
```

TERMINAL (menggunakan algoritma GBFS dengan heuristik Chebyshev)



```
PS E:\Kuliah\Semester_4\Algorithm_Strategy\Tucil3\Tucil3_13524113_13524147> python main.py
>> Heuristik apa yang anda pilih? (H1/H2/H3)
H1=Manhattan | H2=Chebyshev | H3= Euclidean
>>H2

>> Mencari solusi dengan GBFS (H2)...

Solusi yang ditemukan : DR
cost dari solusi : 9

Initial
XXXXX
XZ**X
X*X*X
X**OX
XXXXX

Step 1 :D
XXXXX
X***X
X*X*X
XZ**X
XXXXX

Step 2 :R
XXXXX
X***X
X*X*X
X**OX
XXXXX

>> Waktu eksekusi : 0.12 ms
>> Banyak iterasi : 3 iterasi

>> Apakah anda ingin melakukan playback? (Ya/Tidak): Tidak

>> Apakah anda ingin menyimpan solusi? (Ya/Tidak): Tidak
PS E:\Kuliah\Semester_4\Algorithm_Strategy\Tucil3\Tucil3_13524113_13524147>
```

GUI (menggunakan algoritma UCS)



Ice Sliding Puzzle Solver



X	X	X	X	X
X	Z	*	*	X
X	*	X	*	X
X	*	*	P	X
X	X	X	X	X

Algoritma

☒ UCS

☐ GBFS

☐ A*

Heuristic

☒ H1 - Manhattan

☐ H2 - Chebyshev

☐ H3 - Euclidean

Load Map

Run

Informasi Solusi

Cost: 8

Waktu: 0.13 ms

Iterasi: 4

Solusi: R → D

Export Solution

Playback Solusi

Prev

Next

Jump

Play

Pause

Stop

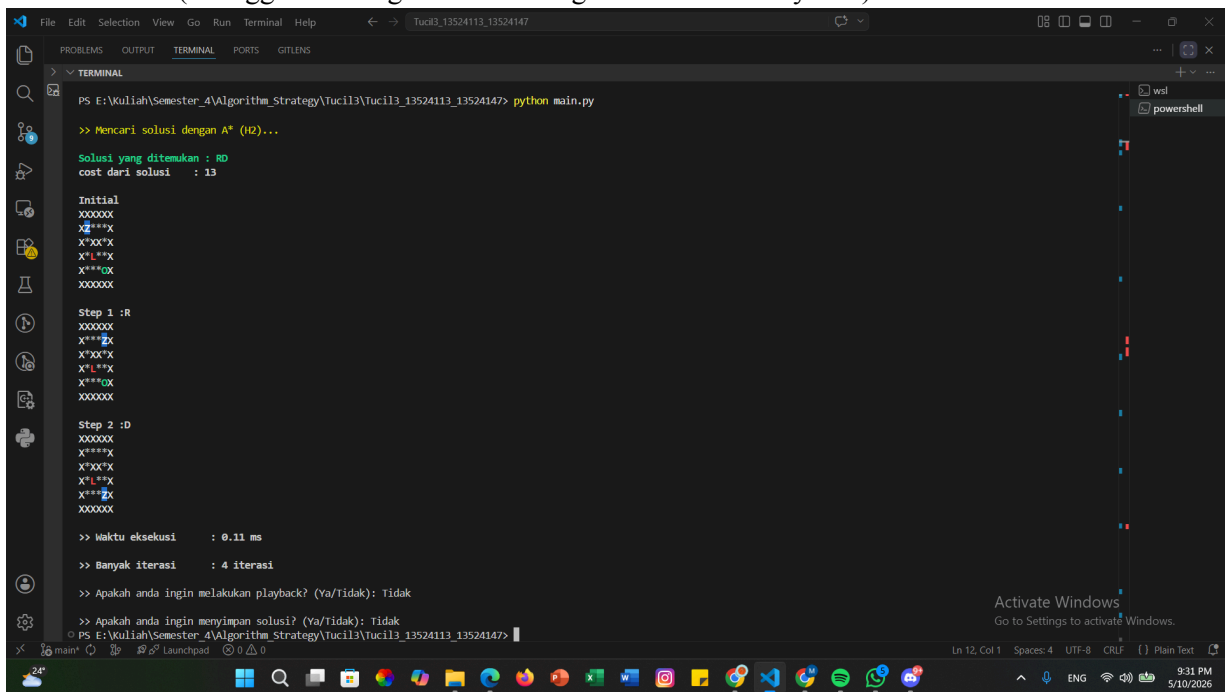
Speed (ms): 500

Step: 2/2

3. Test Case 3

```
6 6
XXXXXX
XZ***X
X*XX*X
X*L**X
X***OX
XXXXXX
999 999 999 999 999 999
999 1 2 3 4 999
999 2 999 999 1 999
999 3 999 2 2 999
999 1 3 4 1 999
999 999 999 999 999 999
```

TERMINAL (menggunakan algoritma A* dengan heuristik Chebyshev)



```
PS E:\Kuliah\Semester_4\Algorithm_Strategy\Tucil3_13524113_13524147> python main.py

>> Mencari solusi dengan A* (H2)...

Solusi yang ditemukan : RD
cost dari solusi : 13

Initial
XXXXXX
XZ***X
X*XX*X
X*L**X
X***OX
XXXXXX

Step 1 :R
XXXXXX
X***X
X*XX*X
X*L**X
X***OX
XXXXXX

Step 2 :D
XXXXXX
X***X
X*XX*X
X*L**X
X***X
XXXXXX

>> Waktu eksekusi : 0.11 ms

>> Banyak iterasi : 4 iterasi

>> Apakah anda ingin melakukan playback? (Ya/Tidak): Tidak

>> Apakah anda ingin menyimpan solusi? (Ya/Tidak): Tidak
PS E:\Kuliah\Semester_4\Algorithm_Strategy\Tucil3_13524113_13524147>
```

GUI (menggunakan algoritma GBFS dengan heuristik Euclidean)



Ice Sliding Puzzle Solver

X	X	X	X	X	X
X	Z	*	*	*	X
X	*	X	X	*	X
X	*	L	*	*	X
X	*	*	*	P	X
X	X	X	X	X	X

Algoritma

- ☐ UCS
☒ GBFS
☐ A*

Heuristic

- ☐ H1 - Manhattan
☐ H2 - Chebyshev
☒ H3 - Euclidean

Load Map

Run

Informasi Solusi

Cost: 14
Waktu: 0.09 ms
Iterasi: 3
Solusi: D → R

Export Solution

Playback Solusi

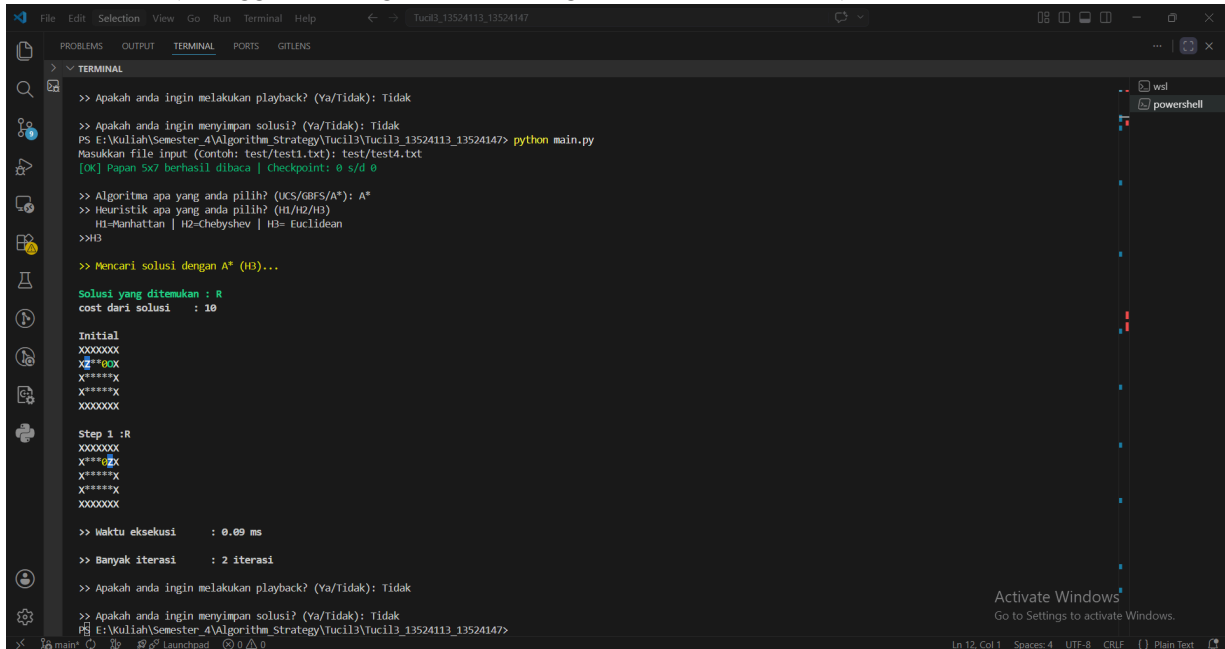
Speed (ms): 500

Step: 2/2

4. Test Case 4

```
5 7
XXXXXXX
XZ**00X
X*****X
X*****X
XXXXXXX
999 999 999 999 999 999 999
999 1 2 3 1 4 999
999 5 2 1 3 2 999
999 1 3 2 1 3 999
999 999 999 999 999 999 999
```

TERMINAL (menggunakan algoritma A* dengan heuristik Euclidean)



```
>> Apakah anda ingin melakukan playback? (Ya/Tidak): Tidak
>> Apakah anda ingin menyimpan solusi? (Ya/Tidak): Tidak
PS E:\Kuliah\Semester_4\Algorithm_Strategy\Tucil3\Tucil3_13524113_13524147> python main.py
Masukkan file input (Contoh: test/test1.txt): test/test4.txt
[OK] Papan 5x7 berhasil dibaca | Checkpoint: 0 s/d 0
>> Algoritma apa yang anda pilih? (UCS/GDFS/A*): A*
>> Heuristik apa yang anda pilih? (H1/H2/H3)
H1=Manhattan | H2=Chebyshev | H3= Euclidean
>>H3
>> Mencari solusi dengan A* (H3)...
Solusi yang ditemukan : R
cost dari solusi : 10
Initial
XXXXXXX
XZ**00X
X*****X
X*****X
XXXXXXX
Step 1 :R
XXXXXXX
X*****X
X*****X
X*****X
XXXXXXX
>> Waktu eksekusi : 0.09 ms
>> Banyak iterasi : 2 iterasi
>> Apakah anda ingin melakukan playback? (Ya/Tidak): Tidak
>> Apakah anda ingin menyimpan solusi? (Ya/Tidak): Tidak
PS E:\Kuliah\Semester_4\Algorithm_Strategy\Tucil3\Tucil3_13524113_13524147>
```

GUI (menggunakan algoritma A* dengan heuristik Manhattan)

Ice Sliding Puzzle Solver

X	X	X	X	X	X	X
X	Z	*	*	0	P	X
X	*	*	*	*	*	X
X	*	*	*	*	*	X
X	X	X	X	X	X	X

Algoritma

- ☐ UCS
☐ GBFS
☒ A*

Heuristic

- ☒ H1 - Manhattan
☐ H2 - Chebyshev
☐ H3 - Euclidean

Load Map

Run

Informasi Solusi

Cost: 10
Waktu: 0.09 ms
Iterasi: 2
Solusi: R

Export Solution

Playback Solusi

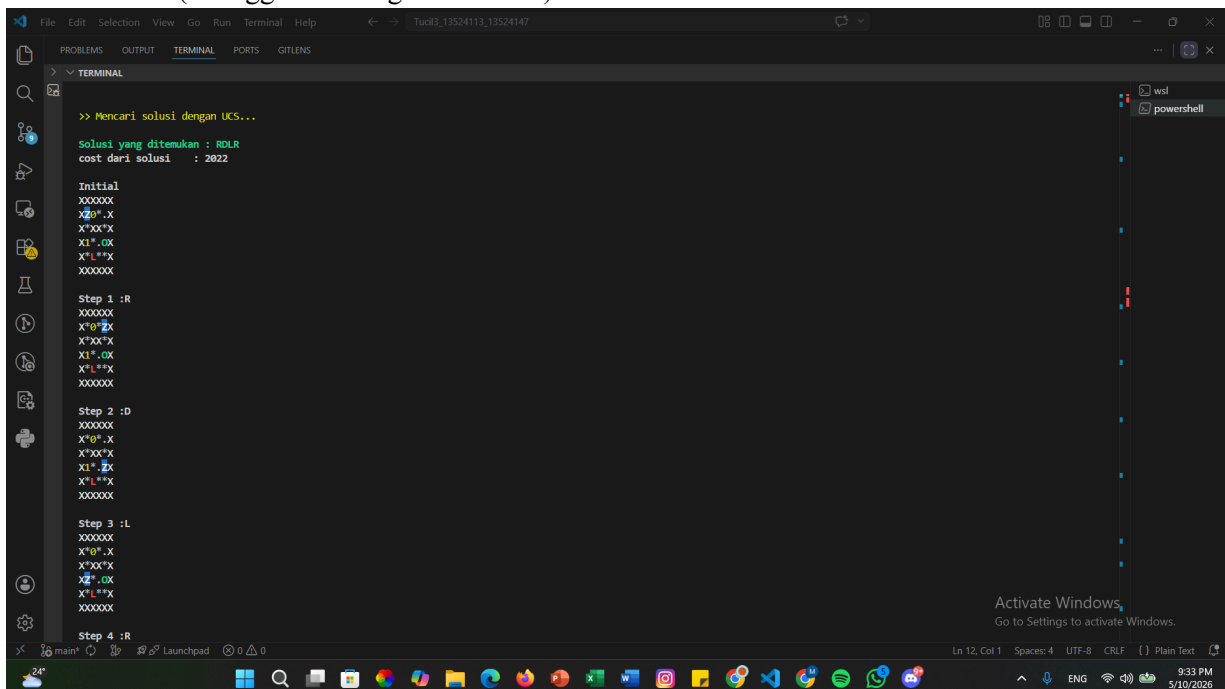
Speed (ms): 500

Step: 1/1

5. Test Case 5

```
6 6
XXXXXX
XZ0*.X
X*XX*X
X1*.OX
X*L**X
XXXXXX
999 999 999 999 999 999
999 5 3 2 999 999
999 8 999 4 999 999
999 7 2 3 1 999
999 9 999 5 6 999
999 999 999 999 999 999
```

TERMINAL (menggunakan algoritma UCS)



```
>> Mencari solusi dengan UCS...
Solusi yang ditemukan : RDLR
cost dari solusi : 2022

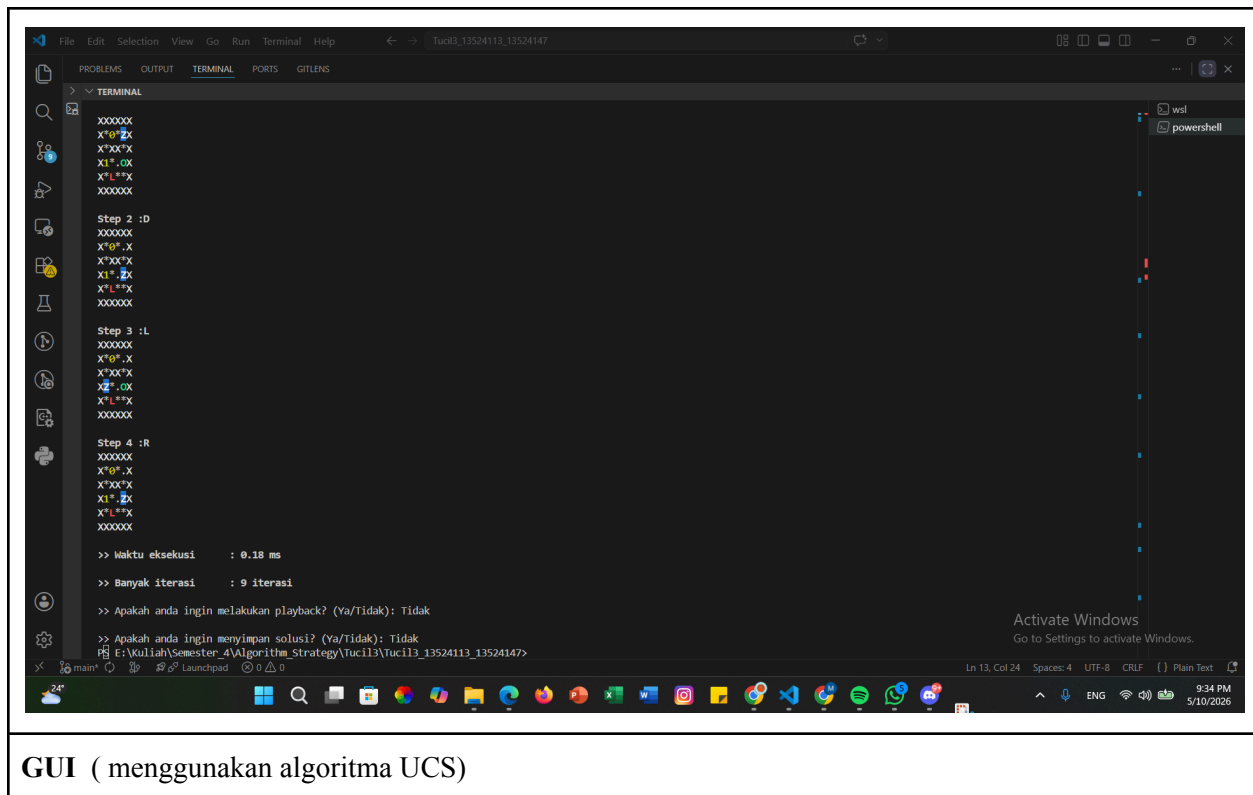
Initial
XXXXXX
XZ0*.X
X*XX*X
X1*.OX
X*L**X
XXXXXX

Step 1 :R
XXXXXX
X*0*.X
X*XX*X
X1*.OX
X*L**X
XXXXXX

Step 2 :D
XXXXXX
X*0*.X
X*XX*X
X1*.X
X*L**X
XXXXXX

Step 3 :L
XXXXXX
X*0*.X
X*XX*X
XZ*.OX
X*L**X
XXXXXX

Step 4 :R
XXXXXX
X*0*.X
X*XX*X
X1*.OX
X*L**X
XXXXXX
```



GUI (menggunakan algoritma UCS)



Ice Sliding Puzzle Solver

X	X	X	X	X	X
X	Z	0	*		X
X	*	X	X	*	X
X	1	*		P	X
X	*	L	*	*	X
X	X	X	X	X	X

Algoritma

- ☒ UCS
☐ GBFS
☐ A*

Heuristic

- ☒ H1 - Manhattan
☐ H2 - Chebyshev
☐ H3 - Euclidean

Load Map

Run

Informasi Solusi

Cost: 2022
Waktu: 0.12 ms
Iterasi: 9
Solusi: R → D → L → R

Export Solution

Playback Solusi

Speed (ms): 500

Step: 4/4

BAB V ANALISIS HASIL PERCOBAAN

Berdasarkan hasil pengujian yang telah dilakukan pada beberapa test case, program Ice Sliding Puzzle Solver berhasil menemukan solusi sesuai aturan permainan menggunakan algoritma UCS, GBFS, dan A*. Seluruh algoritma dapat berjalan dengan baik-baik pada mode terminal maupun GUI.

Algoritma uniform-Cost Search (UCS) dapat menghasilkan solusi optimal karena mempertimbangkan total cost perjalanan. Akan tetapi, waktu yang digunakan juga lebih lama dibandingkan algoritma lain karena UCS melakukan scanning terhadap node secara lebih menyeluruh tanpa menggunakan heuristik.

Algoritma Greedy Best-First Search (GBFS) memiliki performa pencarian yang lebih cepat dibandingkan dengan UCS karena berfokus kepada nilai heuristik menuju goal. Tetapi, solusi yang dihasilkan tidak selalu optimal karena algoritma tidak mempertimbangkan total cost yang telah ditempuh.

Sementara itu, Algoritma A* merupakan algoritma yang paling seimbang karena kecepatan dan optimalitas. Dengan memanfaatkan kombinasi cost aktual dan heuristik, A* mampu menemukan solusi optimal dengan jumlah eksplorasi node yang lebih sedikit dibandingkan UCS.

Berdasarkan pengujian heuristik, Manhattan Distance memberikan estimasi yang lebih cepat stabil dan efektif pada sebagian besar kasus. Chebyshev Distance cenderung lebih cepat karena lebih optimis, sedangkan Euclidean Distance memberikan pendekatan jarak yang cukup baik tetapi terkadang kurang efisien dibanding Manhattan pada struktur grid permainan.

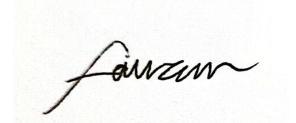
Secara keseluruhan, implementasi program telah memenuhi spesifikasi tugas, mulai dari pembacaan file input, proses pencarian solusi, visualisasi GUI, hingga penyimpanan hasil solusi. Program juga berhasil mengimplementasikan beberapa algoritma pathfinding dan alternatif heuristik sesuai kebutuhan tugas kecil.

LAMPIRAN

Link Github : [Tucil3_13524113_13524147](#)

Pernyataan tidak melakukan kecurangan :

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Fauzan Mohamad Abdul Ghani



Muh. Hartawan Haidir

CheckList Tugas Kecil :

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)		✓